

DevOps实战

一、DevOps概念

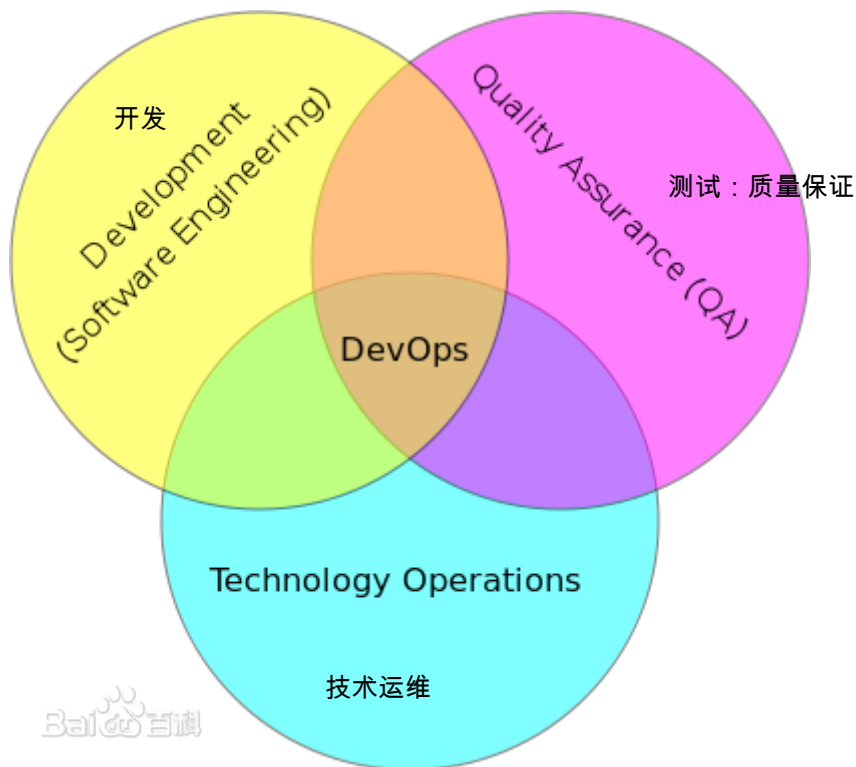
1、DevOps是什么

Development和Operations的组合词；

本地搭建集群

https://mirrors.huaweicloud.com/centos/7/isos/x86_64/

vmware:https://mp.weixin.qq.com/mp/appmsgalbum?__biz=Mzk4ODQ3MzgZnw==&action=getalbum&album_id=3999442668693553169&scene=21#wechat_redirect



DevOps能力环



无尽头的可能性：DevOps涵盖了代码、部署目标的发布和反馈等环节，闭合成一个无限大符号形状的DevOps能力闭环。

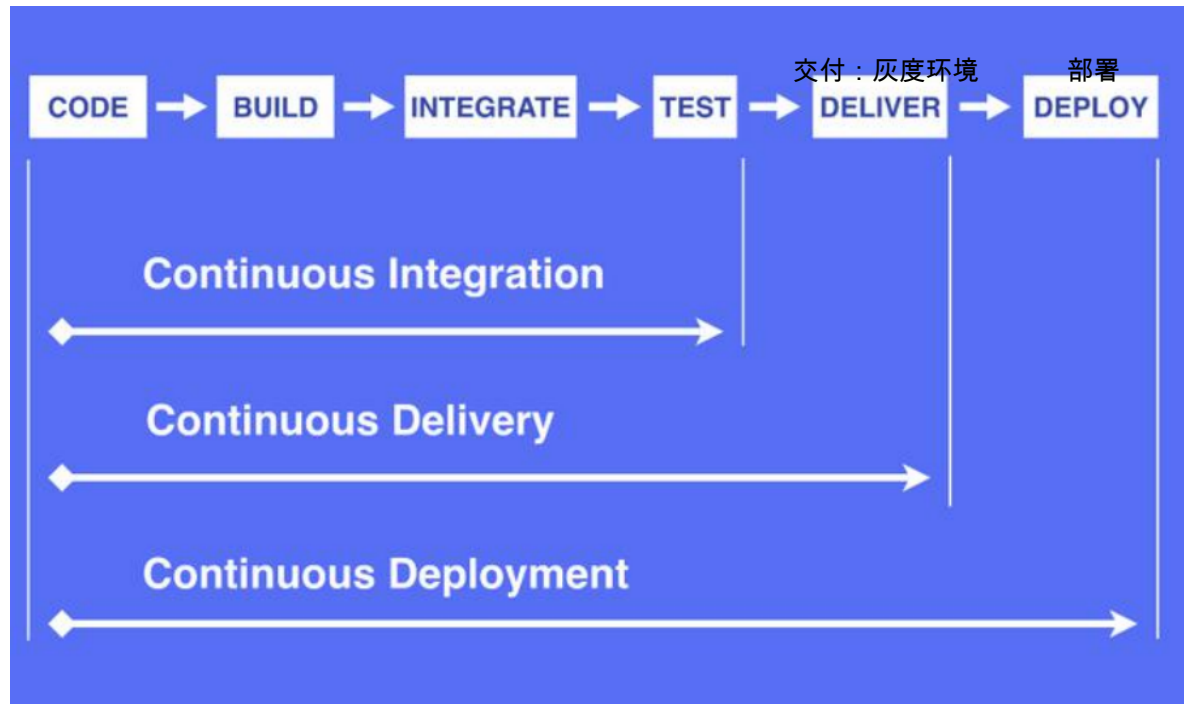
DevOps: Development 和 Operations 的组合

- DevOps 看作开发（软件工程）、技术运营和质量保障（QA）三者的交集。
- 突出重视软件开发人员和运维人员的沟通合作，通过自动化流程来使得软件构建、测试、发布更加快捷、频繁和可靠。
- DevOps 希望做到的是软件产品交付过程中 **IT 工具链的打通**，使得各个团队减少时间损耗，更加高效地协同工作。专家们总结出了下面这个 DevOps 能力图，良好的闭环可以大大 增加整体的产出。

2、CICD是什么

持续集成 持续部署

1、基本概念



1、持续集成 (Continuous Integration)

持续集成是指软件个人研发的部分向软件整体部分交付，频繁进行集成以便更快地发现其中的错误。“持续集成”源自于极限编程（XP），是 XP 最初的 12 种实践之一。

CI 需要具备这些：

- **全面的自动化测试。**这是实践持续集成&持续部署的基础，同时，选择合适的自动化测试工具也极其重要；
- **灵活的基础设施。**容器，虚拟机的存在让开发人员和 QA 人员不必再大费周折；
- **版本控制工具。**如 Git, CVS, SVN 等；
- **自动化的构建和软件发布流程的工具，**如 Jenkins, flow.ci；
- **反馈机制。**如构建/测试的失败，可以快速地反馈到相关负责人，以尽快解决达到一个更稳定的版本。

2、持续交付 (Continuous Delivery)

持续交付在持续集成的基础上，将集成后的代码部署到**更贴近真实运行环境的「类生产环境」**（production-like environments）中。持续交付优先于整个产品生命周期的软件部署，建立在高水平自动化持续集成之上。

灰度发布。

持续交付和持续集成的优点非常相似：

- **快速发布。**能够应对业务需求，并更快地实现软件价值。
- **编码->测试->上线->交付**的频繁迭代周期缩短，同时获得迅速反馈；
- **高质量的软件发布标准。**整个交付过程标准化、可重复、可靠，
- **整个交付过程进度可视化，**方便团队人员了解项目成熟度；

- **更先进的团队协作方式。**从需求分析、产品的用户体验到交互设计、开发、测试、运维等角色密切协作，相比于传统的瀑布式软件团队，更少浪费。

3、持续部署 (Continuous Deployment)

持续部署是指当交付的代码通过评审之后，**自动部署到生产环境中**。持续部署是持续交付的最高阶段。这意味着，所有通过了一系列的自动化测试的改动都将自动部署到生产环境。它也可以被称为“Continuous Release”。

“开发人员**提交代码**，持续集成服务器获取代码，执行单元测试，根据测试结果决定是否部署到预演环境，如果成功部署到预演环境，进行整体**验收测试**，如果**测试通过**，**自动部署到产品环境**，**全程自动化高效运转**。

持续部署主要好处是，可以相对独立地部署新的功能，并能快速地收集真实用户的反馈。

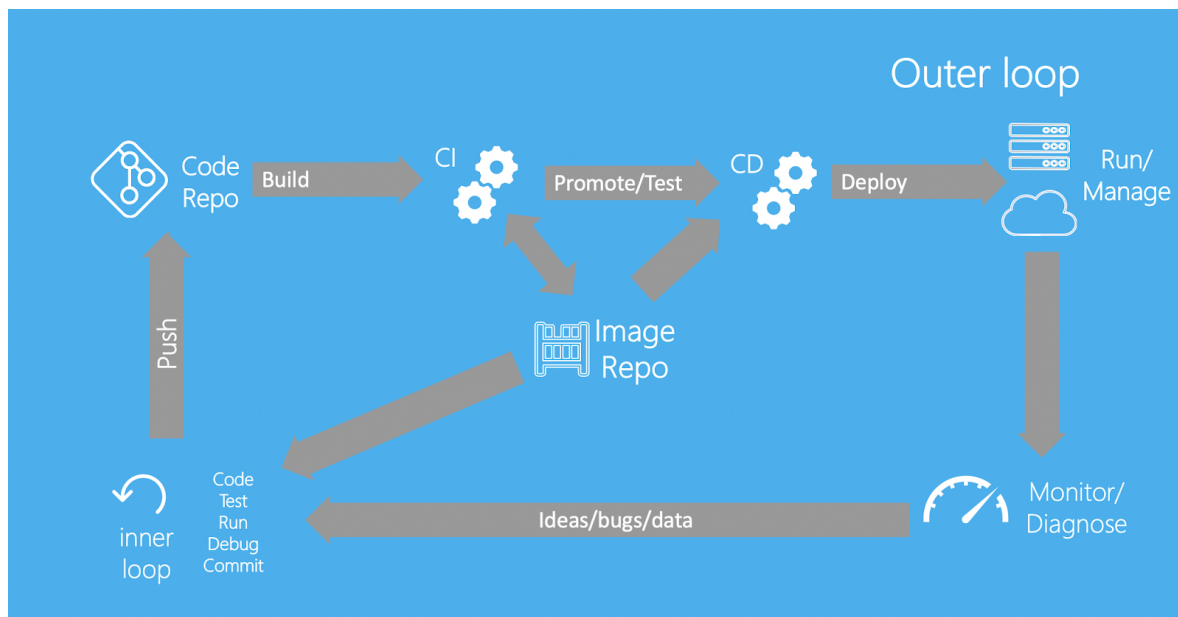
“You build it, you run it”，这是 **Amazon** 一年可以完成 **5000 万次部署**，平均每个工程师每天部署超过 50 次的核心秘籍。

$5000/365 = 15$ 万次

开发人员代码敲完。可以release的时候，提交代码，剩下的全部一站式自动搞定

2、最佳实践

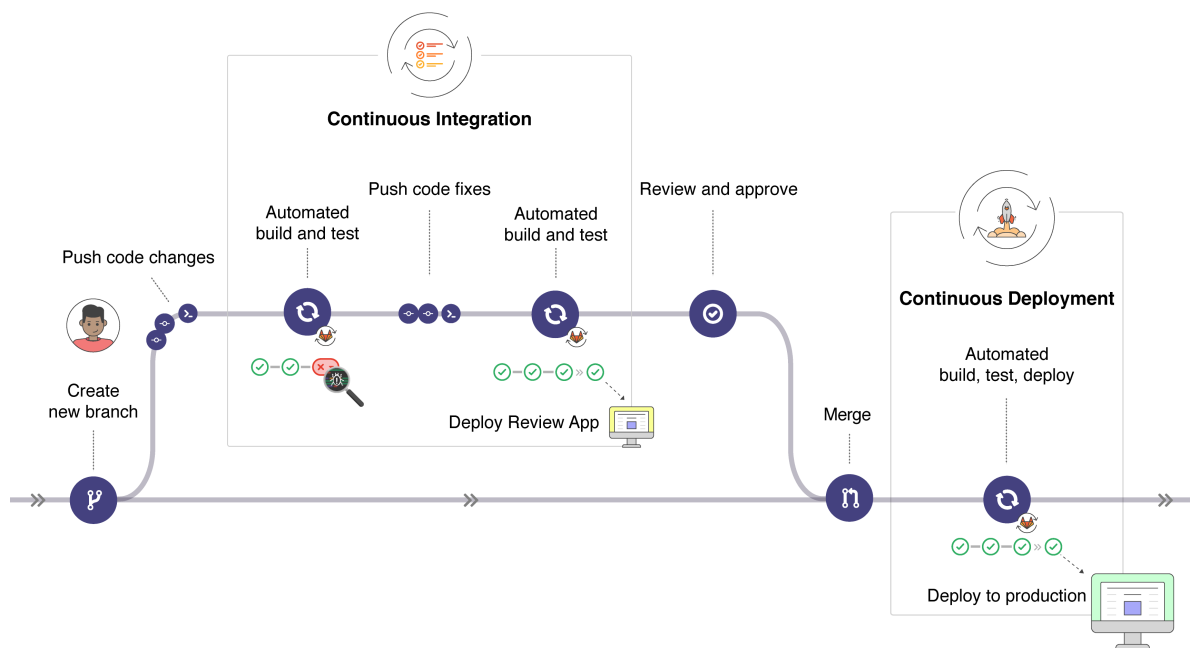
1、内循环与外循环



- 内循环 (开发要做的事情) :
 - 编码、测试、运行、debug、提交
- 代码推送到代码仓库 (svn, git) 【代码回滚】
- 进行CI过程 (持续集成) , 万物皆可容器化。打包成一个Docker镜像
- 镜像推送到镜像仓库
- 测试
- 持续部署流程 (CD) , 拿到之前的镜像, 进行CD。怎么放到各种环境: uat、test、prod
- 外循环 ()
 - 运行时监控

- 生产环境的管理
- 监控
- 线上反馈到开发
- 来到内循环

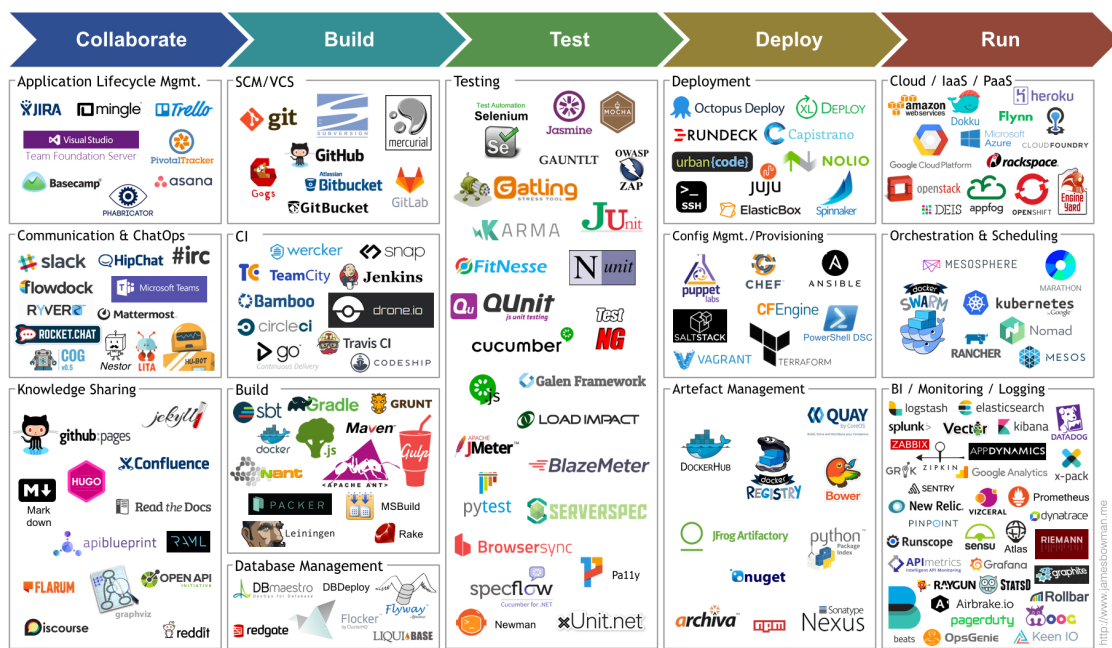
2、实践流程



新功能，bug修复。

- 创建分支来做这个事情（开发功能）
- 提交分支的代码改变
- 进入持续集成流程
 - 当前分支代码功能性自动化构建和测试
 - 自动工具推送这次提交
 - 自动化集成测试
 - 可以看到效果
 - 人工确认此次功能是否发布到生产环境
- 代码合并。
- 进入持续部署流程
 - 构建、测试、发布.....

3、CICD LandSpace



二、Jenkins

<https://www.jenkins.io/zh/doc/book/installing/#docker>

```
1  /var/jenkins_home    jenkins的家目录
2  包含了jenkins的所有配置。
3
4
5  以后要注意备份，非常重要    /var/jenkins_home    （以文件的方式固化的）
```

Jenkins也是Java写的

Jenkins镜像用 [jenkinsci/blueocean](https://jenkinsci.github.io/blueocean)

Jenkins能驱动我们整个CICD过程的很多工具



Jenkins

构建伟大，无所不能

Jenkins是开源CI&CD软件领导者，提供超过1000个插件来支持构建、部署、自动化，满足任何项目的需要。

文档

下载

1、Jenkins安装

<https://www.jenkins.io/zh/doc/book/installing/>

```
1  docker run \  
2    -u root \  
3    -d \  
4    -p 8080:8080 \  
5    -p 50000:50000 \  
6    -v jenkins-data:/var/jenkins_home \  
7    -v /etc/localtime:/etc/localtime:ro \  
8    -v /var/run/docker.sock:/var/run/docker.sock \  
9    --restart=always \  
10   jenkinsci/blueocean  
11  
12  
13   #自己构建镜像 RUN的时候就把时区设置好  
14   #如果是别人的镜像, docker hub, UTC; 容器运行时, -v  
15   /etc/localtime:/etc/localtime:ro  
16  
17  
18   jenkinsci/jenkins 是没有 blueocean插件的, 得自己装  
19   jenkinsci/blueocean: 带了的  
20  
21   #/var/run/docker.sock 表示Docker守护程序通过其监听的基于Unix的套接字。 该映射允许  
22   jenkinsci/blueocean 容器与Docker守护进程通信, 如果 jenkinsci/blueocean 容器需要实例化  
23   其他Docker容器, 则该守护进程是必需的。 如果运行声明式管道, 其语法包含agent部分用 docker; 例  
    如, agent { docker { ... } } 此选项是必需的。
```

安装插件, 并配置用户

创建第一个管理员用户

用户名:	lfy
密码:	
确认密码:	
全名:	leifengyang
电子邮件地址:	534096094@qq.com

```
1  #备份jenkins
2  tar -cvf jenkins_data.tar /var/lib/docker/volumes/jenkins-data/_data/
3
4  #恢复jenkins
5  tar -xvf jenkins_data.tar /var/lib/docker/volumes/jenkins-data/_data/
```

2、Jenkins实战

代码在本地修改---提交到远程gitee---触发jenkins整个自动化构建流程（打包，测试，发布，部署）

1、准备一个git项目进行测试

我们以gitee为例，github可能太慢了。需要idea安装gitee插件。或者自己熟悉手动命令也行。

步骤：

- 1、idea创建Spring Boot项目
- 2、VCS - 创建git 仓库
- 3、gitee创建一个空仓库，示例为public
- 4、idea提交内容到gitee
- 5、开发项目基本功能，并在项目中创建一个Jenkinsfile文件
- 6、创建一个名为 devops-java-demo的流水线项目，使用项目自己的流水线

<https://www.jenkins.io/zh/doc/book/pipeline/getting-started/>

Jenkins的工作流程

1、先定义一个流水线项目，指定项目的git位置

私有仓库搞个Credentials登录即可

- 流水线启动
 - 1、先去git位置自动拉取代码
 - 2、解析拉取代码里面的Jenkinsfile文件
 - 3、按照Jenkinsfile指定的流水线开始加工项目

0、jenkins的家目录 /var/jenkins_home 已经被我们docker外部挂载了 ；

/var/lib/docker/volumes/jenkins-data/_data

1、WORKSPACE（工作空间）=/var/jenkins_home/workspace/java-devops-demo

- 每一个流水线项目，占用一个文件夹位置
- BUILD_NUMBER=5；当前第几次构建
- WORKSPACE_TMP（临时目录）=/var/jenkins_home/workspace/java-devops-demo@tmp

1

1

1

2、常用的环境如果没有，jenkins配置环境一大堆操作

3、JENKINS_URL： <http://139.198.27.103:8080/>；就是jenkins服务器的地址

<http://lpgitee:1111c601fa5339441452e63f299fd7fb66@3fa5b436.r38.cpolar.top/job/java-devops-demo/build?token=lpruoyu>

<http://giteetestuser:11dde9be0421613d7c0bd6dbef3f154b29@3fa5b436.r38.cpolar.top/job/java-devops-demo/build?token=lpruoyu>

2、远程构建触发

http://用户:令牌@jenkins地址/job/java-devops-demo/build?token=TOKEN_NAME

期望效果：远程的github代码提交了，jenkins流水线自动触发构建。

实现流程：

这块的令牌是在jenkins中给用户添加的API Token 【在用户->Security页面配置】

- 1、保证jenkins所在主机能被远程访问
- 2、jenkins中远程触发需要权限，我们应该使用用户进行授权
- 3、配置gitee/github，webhook进行触发

- 1 #远程构建即使配置了github 的webhook，默认会403.我们应该使用用户进行授权
- 2 1、创建一个用户
- 3 2、一定随便登陆激活一次
- 4 3、生成一个apitoken
- 5 <http://leifengyang:113620edce6200b9c78ecadb26e9cf122e@139.198.186.134:8080/job/devops-java-demo/build?token=leifengyang>
- 6

☒ 触发远程构建 (例如,使用脚本)

身份验证令牌

Use the following URL to trigger build remotely: JENKINS_URL/job/simple-java-maven-app/build?token=TOKEN_NAME 或者 /buildWithParameters?token=TOKEN_NAME
Optionally append &cause=Cause+Text to provide text that will be included in the recorded build cause.

远程触发： [JENKINS_URL/job/simple-java-maven-app/build?token=](JENKINS_URL/job/simple-java-maven-app/build?token=TOKEN_NAME) [TOKEN_NAME](#) 请求即可

6、流水线语法

<https://www.jenkins.io/zh/doc/book/pipeline/>

1、基础格式

```

1 pipeline {
2     agent any
3     environment {
4         CC = 'clang'
5     }
6     stages {
7         stage('Example') {
8             steps {
9                 sh 'printenv'
10                sh 'echo $CC'
11            }
12        }
13    }
14 }

```

2、环境变量

3、密钥管理

```
// environment aa = credxxx("xxxxx")
```

4、自定义agent

可更新 可选插件 已安装 高级

Install ↑	Name
☐	Docker 云提供商 集群管理和分布式构建 docker This plugin integrates Jenkins with Docker
☐	Docker Commons api-plugin docker 插件库（被其他插件使用） Provides the common shared functionality for various Docker-related plugins.
☒	Docker Pipeline 开发 DevOps docker pipeline Build and use Docker containers from pipelines.
	Docker API

```

1 //需要安装docker pipeline插件
2
3 pipeline {
4     agent none

```

```

5     stages {
6         stage('Example Build') {
7             agent {
8                 docker 'maven:3-alpine'
9                 //args 是指定 docker run 的所有指令
10                args '-v /var/jenkins_home/maven/.m2:/root/.m2'
11            }
12            steps {
13                echo 'Hello, Maven'
14                sh 'mvn --version'
15            }
16        }
17        stage('Example Test') {
18            agent { docker 'openjdk:8-jre' }
19            steps {
20                echo 'Hello, JDK'
21                sh 'java -version'
22            }
23        }
24    }
25 }

```

<https://hub.docker.com/layers/library/maven/3-alpine/images/sha256-5435658a63acb615bd9ab5d9249be37de8a1b4e996a6996e26a5127e2a14e79c> ; 发现3-alpine的maven版本是maven-3/3.5.0/

- 配置maven加速

- 把Maven的配置文件放在jenkins-data里面的某个位置

<https://github.com/apache/maven/releases/tag/maven-3.5.0>中的settings.xml复制过来

```

1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <settings xmlns="http://maven.apache.org/SETTINGS/1.0.0"
4            xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5            xsi:schemaLocation="http://maven.apache.org/SETTINGS/1.0.0
6            http://maven.apache.org/xsd/settings-1.0.0.xsd">
7      <!-- localRepository
8       | The path to the local repository maven will use to store artifacts.
9       |
10      | Default: ${user.home}/.m2/repository
11      <localRepository>H:\Devsoft\apache-maven-3.6.1\repository</localRepository>
12      用户目录下的.m2是所有jar包的地方；maven容器内jar包的位置
13      -->
14      <localRepository>/root/.m2</localRepository>
15
16
17      <pluginGroups>
18
19      </pluginGroups>
20
21      <proxies>
22
23      </proxies>
24
25      <servers>
26
27      </servers>
28
29
30      <mirrors>

```



```

31     <mirror>
32         <id>nexus-aliyun</id>
33         <mirrorOf>central</mirrorOf>
34         <name>Nexus aliyun</name>
35         <url>http://maven.aliyun.com/nexus/content/groups/public</url>
36     </mirror>
37 </mirrors>
38
39 <profiles>
40     <profile>
41         <id>jdk-1.8</id>
42         <activation>
43             <activeByDefault>true</activeByDefault>
44             <jdk>1.8</jdk>
45         </activation>
46         <properties>
47             <maven.compiler.source>1.8</maven.compiler.source>
48             <maven.compiler.target>1.8</maven.compiler.target>
49
50         <maven.compiler.compilerVersion>1.8</maven.compiler.compilerVersion>
51         </properties>
52     </profile>
53 </profiles>
54 </settings>

```

默认所有的可变配置项都推荐放在jenkins-home的位置，增强移植性

如何自定义使用maven的配置文件

```

1  //流水线这样
2  stage('编译'){
3      agent {
4          docker {
5              image 'maven:3-alpine' //用完就会杀掉
6              args '-v maven-repo:/root/.m2'
7              //docker run -v repo:/root/.m2 这个容器是直接linux上的,docker
8              volume也很灵活
9          }
10     }
11     steps {
12         sh 'pwd && ls -alh'
13         sh 'mvn -v'
14         sh 'mvn clean package -s
15         "/var/jenkins_home/appconfig/maven/settings.xml" -Dmaven.test.skip=true '
16         //残留的问题，下一次运行这个命令，已经之前下载的jar还会下载。
17     }
18 }
19
20 //再给 linux主机挂载的jenkins_home位置自定义一个 settings.xml 即可
21
22 //原理: jenkins在解析流水线期间，可以任意访问jenkins家目录的位置和相关环境信息
23
24 //一个配置文件+docker maven agent = maven加速

```

- 临时容器导致的问题

- 1、第一次检出代码，默认在 /var/jenkins_home/workspace/【java-devops-demo】
- 2、使用docker临时agent的时候，每一个临时容器运行又分配临时目录 /var/jenkins_home/workspace/java-devops-demo@2；临时目录默认就是workspace/java-devops-demo 的内容
- 3、在临时容器里面 运行的mvn package命令，会在 /var/jenkins_home/workspace/java-devops-demo@2 进行工作
- 4、package到了 /var/jenkins_home/workspace/java-devops-demo@2 位置
- 5、进入下一步【新的stage】进行打包镜像，又会回到 /var/jenkins_home/workspace /【java-devops-demo】这个默认位置；也就是说：每一个 stage的开始，都会重置到默认的WORKSPACE位置
- 6、这个位置没有运行过 mvn clean package，没有target。默认的工作目录 没有 target

5、参数化

6、条件表达式

```

1  ##默认的when
2
3  ##还可以写脚本哦。。。
4
```

7、修改jenkins插件源

```

1
2
```

jenkins插件中心的插件；



8、推荐安装的插件

- Docker Pipeline
 - 安装Docker Pipeline会自动安装docker相关的
 - 这个允许我们自定义agent使用docker环境
- Git Parameter
 - 解析git参数，允许我们选择分支进行构建
- Active Choices
 - 可以做到参数的级联选择
- **Generic Webhook Trigger**
 - 通用的webhook触发器，构建更强大的webhook功能
- Role-based Authorization Strategy
 - RBAC权限指定，给一个用户精确指定权限
- List Git Branches Parameter
 - 列出分支参数
- **Build With Parameters**
 - 基于自定义参数构建

我的插件列表



Jenkins

🔍 查找

🔔 1

👤 admin

🚪 注销

Dashboard > 插件管理

🏠 返回工作台

⚙️ 管理 Jenkins

🌱 更新中心

🔍 filter

可更新 可选插件 已安装 高级

启用	名称 ↓	版本	上一个安装的版本	卸载
☒	Active Choices Plug-in This plug-in provides additional parameter types for jobs, that allow you to cascade changes and render images or other HTML elements instead of the traditional parameter.	2.5.6		卸载
☒	Ant Plugin Adds Apache Ant support to Jenkins	1.11		卸载
☒	Apache HttpComponents Client 4.x API Plugin Bundles Apache HttpComponents Client 4.x and allows it to be used by Jenkins plugins. This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more information.	4.5.13-1.0		卸载
☒	Authentication Tokens API Plugin This plugin provides an API for converting credentials into authentication tokens in Jenkins.	1.4		卸载
☒	Autofavorite for Blue Ocean Automatically favorites multibranch pipeline jobs when user is the author	1.2.4		卸载
☒	Bitbucket Branch Source Plugin Allows to use Bitbucket Cloud and Bitbucket Server as sources for multi-branch projects. It also provides the required connectors for Bitbucket Cloud Team and Bitbucket Server Project folder (also known as repositories auto-discovering). This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more information.	2.9.8		卸载
☒	Bitbucket Pipeline for Blue Ocean BlueOcean Bitbucket pipeline creator	1.24.6		卸载
☒	Blue Ocean BlueOcean Aggregator	1.24.6		卸载
☒	Blue Ocean Core JS The Jenkins Plugins Parent POM Project	1.24.6		卸载
☒	Blue Ocean Pipeline Editor The Blue Ocean Pipeline Editor is the simplest way for anyone wanting to get started with creating Pipelines in Jenkins	1.24.6		卸载
☒	Bootstrap 4 API Plugin Provides Bootstrap 4 for Jenkins plugins.	4.6.0-3		卸载
☒	bouncycastle API This plugin provides an stable API to Bouncy Castle related tasks.	2.16.0		卸载
☒	Branch API Plugin This plugin provides an API for multiple branch based projects.	2.6.3		卸载
☒	Build Timeout This plugin allows builds to be automatically terminated after the specified amount of time has elapsed. This plugin is up for adoption! We are looking for new maintainers. Visit our Adopt a Plugin initiative for more information.	1.20		卸载
☒	Build With Parameters Allows the user to provide parameters for a build in the url (similar to /job/JOBNAME/buildWithParameters), prompting for confirmation before triggering the job.	1.5.1		卸载
☒	Checks API plugin This plugin defines an API for Jenkins to publish checks to SCM platforms.	1.7.0		卸载
☒	Command Agent Launcher Allows agents to be launched using a specified command.	1.2		卸载
☒	Common API for Blue Ocean This plugin is a part of Blue Ocean UI	1.24.6		卸载
☒	Config API for Blue Ocean BlueOcean Analytics Tools plugin	1.24.6		卸载
☒	Credentials Binding Plugin Allows credentials to be bound to environment variables for use from miscellaneous build steps.	1.24		卸载
☒	Credentials Plugin	2.3.17		卸载

	This plugin allows you to store credentials in Jenkins.		卸载
✓	Dashboard for Blue Ocean Blue Ocean Dashboard	1.24.6	卸载
✓	Design Language The Jenkins Plugins Parent POM Project	1.24.6	卸载
✓	Display URL API Provides the DisplayURLProvider extension point to provide alternate URLs for use in notifications	2.3.4	卸载
✓	Display URL for Blue Ocean This plugin generates BlueOcean specific URLs for the Display URL plugin.	2.4.1	卸载
✓	Docker Commons Plugin Provides the common shared functionality for various Docker-related plugins.	1.17	卸载
✓	Docker Pipeline Build and use Docker containers from pipelines.	1.26	卸载
✓	Durable Task Plugin Library offering an extension point for processes which can run outside of Jenkins yet be monitored.	1.35	卸载
✓	ECharts API Plugin Provides ECharts for Jenkins plugins.	5.0.2-1	卸载
✓	Email Extension Plugin This plugin is a replacement for Jenkins's email publisher. It allows to configure every aspect of email notifications: when an email is sent, who should receive it and what the email says	2.82	卸载
✓	Events API for Blue Ocean Blue Ocean Events	1.24.6	卸载
✓	Favorite This plugin allows users to favorite a job.	2.3.3	卸载
✓	Folders Plugin This plugin allows users to create "folders" to organize jobs. Users can define custom taxonomies (like by project type, organization type etc). Folders are nestable and you can define views within folders. Maintained by CloudBees, Inc.	6.15	卸载
✓	Font Awesome API Plugin Provides the free fonts of Font Awesome for Jenkins plugins.	5.15.2-2	卸载
✓	Generic Webhook Trigger Plugin Can receive any HTTP request, extract any values from JSON or XML and trigger a job with those values available as variables. Works with GitHub, GitLab, Bitbucket, Jira and many more.	1.72	卸载
✓	Git client plugin Utility plugin for Git support in Jenkins	3.7.1	卸载
✓	Git Parameter Plug-In Adds ability to choose branches, tags or revisions from git repositories configured in project.	0.9.13	卸载
✓	Git Pipeline for Blue Ocean BlueOcean Git SCM pipeline creator	1.24.6	卸载
✓	Git plugin This plugin integrates Git with Jenkins.	4.7.1	卸载
✓	GIT server Plugin Allows Jenkins to act as a Git server.	1.9	卸载
✓	GitHub API Plugin This plugin provides GitHub API for other plugins.	1.123	卸载
✓	GitHub Branch Source Plugin Multibranch projects and organization folders from GitHub. Maintained by CloudBees, Inc.	2.10.2	卸载
✓	GitHub Pipeline for Blue Ocean BlueOcean GitHub organization pipeline creator	1.24.6	卸载
✓	GitHub plugin This plugin integrates GitHub to Jenkins.	1.33.1	卸载
✓	Gradle Plugin This plugin allows Jenkins to invoke Gradle build scripts directly.	1.36	卸载
✓	Handy Uri Templates 2.x API Plugin Bundles Handy Uri Templates 2.x and allows it to be used by Jenkins plugins	2.1.8-1.0	卸载
✓	HTML Publisher plugin This plugin publishes HTML reports.	1.25	卸载
✓	i18n for Blue Ocean Blue Ocean Internationalization (i18n) Plugin. This plugin is a part of the Blue Ocean Plugin set.	1.24.6	卸载
✓	Jackson 2 API Plugin This plugin exposes the Jackson 2 JSON APIs to other Jenkins plugins.	2.12.2	卸载
✓	Java JSON Web Token (JWT) Plugin Bundles the Java JSON Web Token (JWT) library.	0.11.2-9.c8b45b8bb173	卸载

☒	JavaScript GUI Lib: ACE Editor bundle plugin JavaScript GUI Lib: ACE Editor bundle plugin.	1.1	<button>卸载</button>
☒	JavaScript GUI Lib: Handlebars bundle plugin JavaScript GUI Lib: Handlebars bundle plugin.	3.0.8	<button>卸载</button>
☒	JavaScript GUI Lib: Moment.js bundle plugin JavaScript GUI Lib: Moment.js bundle plugin.	1.1.1	<button>卸载</button>
☒	jQuery plugin This plugin provides an stable version of jQuery Javascript Library	1.12.4-1	<button>卸载</button>
☒	jQuery3 API Plugin Provides jQuery 3.x for Jenkins plugins.	3.6.0-1	<button>卸载</button>
☒	JSch dependency plugin Jenkins plugin that brings the JSch library as a plugin dependency, and provides an SSHAuthenticatorFactory for using JSch with the ssh-credentials plugin.	0.155.2	<button>卸载</button>
☒	JUnit Plugin Allows JUnit-format test results to be published.	1.49	<button>卸载</button>
☒	JWT for Blue Ocean BlueOcean JWT plugin: Enables JWT based BlueOcean API authentication	1.24.6	<button>卸载</button>
☒	LDAP Plugin Adds LDAP authentication to Jenkins	2.5	<button>卸载</button>
☒	List Git Branches Parameter Plugin Adds ability to choose from git repository revisions or tags	0.0.9	<button>卸载</button>
☒	Localization Support Plugin Supporting infrastructure for standalone localization plugins. On its own, this plugin doesn't do a lot, it requires other plugins to work.	1.1	<button>卸载</button>
☒	Localization: Chinese (Simplified) Jenkins Core 及其插件的简体中文语言包, 由 Jenkins 中文社区 维护。	1.0.24	<button>卸载</button>
☒	Lockable Resources plugin This plugin allows to define external resources (such as printers, phones, computers) that can be locked by builds. If a build requires an external resource which is already locked, it will wait for the resource to be free.	2.10	<button>卸载</button>
☒	Mailer Plugin This plugin allows you to configure email notifications for build results	1.34	<button>卸载</button>
☒	Matrix Authorization Strategy Plugin Offers matrix-based security authorization strategies (global and per-project).	2.6.6	<button>卸载</button>
☒	Matrix Project Plugin Multi-configuration (matrix) project type.	1.18	<button>卸载</button>
☒	OkHttp Plugin This plugin provides OkHttp for other plugins.	3.14.9	<button>卸载</button>
☒	Oracle Java SE Development Kit Installer Allows the Oracle Java SE Development Kit (JDK) to be installed via download from Oracle's website.	1.0	<button>卸载</button>
☒	OWASP Markup Formatter Plugin Uses the OWASP Java HTML Sanitizer to allow safe-seeming HTML markup to be entered in project descriptions and the like.	2.1	<button>卸载</button>
☒	PAM Authentication plugin Adds Unix Pluggable Authentication Module (PAM) support to Jenkins	1.6	<button>卸载</button>
☒	Personalization for Blue Ocean Blue Ocean Personalization	1.24.6	<button>卸载</button>
☒	Pipeline A suite of plugins that lets you orchestrate automation, simple or complex. See Pipeline as Code with Jenkins for more details.	2.6	<button>卸载</button>
☒	Pipeline Graph Analysis Plugin Provides a REST API to access pipeline and pipeline run data.	1.10	<button>卸载</button>
☒	Pipeline implementation for Blue Ocean This plugin is a part of BlueOcean Plugin	1.24.6	<button>卸载</button>
☒	Pipeline SCM API for Blue Ocean This plugin is a part of BlueOcean Plugin	1.24.6	<button>卸载</button>
☒	Pipeline: API Plugin that defines Pipeline API.	2.42	<button>卸载</button>
☒	Pipeline: Basic Steps Commonly used steps for Pipelines.	2.23	<button>卸载</button>
☒	Pipeline: Build Step Adds the Pipeline step <code>build</code> to trigger builds of other jobs.	2.13	<button>卸载</button>
☒	Pipeline: Declarative An opinionated, declarative Pipeline.	1.8.4	<button>卸载</button>

✓	Pipeline: Declarative Extension Points API APIs for extension points used in Declarative Pipelines.	1.8.4	卸载
✓	Pipeline: GitHub Groovy Libraries Allows Pipeline Groovy libraries to be loaded on the fly from GitHub.	1.0	卸载
✓	Pipeline: Groovy Pipeline execution engine based on continuation passing style transformation of Groovy scripts.	2.90	卸载
✓	Pipeline: Input Step Adds the Pipeline step <code>input</code> to wait for human input or approval.	2.12	卸载
✓	Pipeline: Job Defines a new job type for pipelines and provides their generic user interface.	2.40	卸载
✓	Pipeline: Milestone Step Plugin that provides the milestone step	1.3.2	卸载
✓	Pipeline: Model API Model API for Declarative Pipeline.	1.8.4	卸载
✓	Pipeline: Multibranch Enhances Pipeline plugin to handle branches better by automatically grouping builds from different branches.	2.23	卸载
✓	Pipeline: Nodes and Processes Pipeline steps locking agents and workspaces, and running external processes that may survive a Jenkins restart or agent reconnection.	2.38	卸载
✓	Pipeline: REST API Plugin Provides a REST API to access pipeline and pipeline run data.	2.19	卸载
✓	Pipeline: SCM Step Adds a Pipeline step to check out or update working sources from various SCMs (version control).	2.12	卸载
✓	Pipeline: Shared Groovy Libraries Shared libraries for Pipeline scripts.	2.18	卸载
✓	Pipeline: Stage Step Adds the Pipeline step <code>stage</code> to delineate portions of a build.	2.5	卸载
✓	Pipeline: Stage Tags Metadata Library plugin for Pipeline stage tag metadata.	1.8.4	卸载
✓	Pipeline: Stage View Plugin Pipeline Stage View Plugin.	2.19	卸载
✓	Pipeline: Step API API for asynchronous build step primitive.	2.23	卸载
✓	Pipeline: Supporting APIs Common utility implementations to build Pipeline Plugin	3.8	卸载
✓	Plain Credentials Plugin Allows use of plain strings and files as credentials.	1.7	卸载
✓	Plugin Utilities API Plugin Provides several utility classes that can be used to accelerate plugin development.	2.1.0	卸载
✓	Popperjs API Plugin Provides Popper.js for Jenkins plugins.	1.16.1-2	卸载
✓	Pub-Sub "light" Bus A simple Publish-Subscribe light-weight event bus for Jenkins	1.13	卸载
✓	Resource Disposer Plugin Dispose resources asynchronously. Utility plugin for resources that require more retries or take a long time to delete.	0.15	卸载
✓	REST API for Blue Ocean This plugin is a part of Blue Ocean UI	1.24.6	卸载
✓	REST Implementation for Blue Ocean This plugin is a part of Blue Ocean UI	1.24.6	卸载
✓	Role-based Authorization Strategy Enables user authorization using a Role-Based strategy. Roles can be defined globally or for particular jobs or nodes selected by regular expressions.	3.1.1	卸载
✓	SCM API Plugin This plugin provides a new enhanced API for interacting with SCM systems.	2.6.4	卸载
✓	Script Security Plugin Allows Jenkins administrators to control what in-process scripts can be run by less-privileged users.	1.76	卸载
✓	Server Sent Events (SSE) Gateway Plugin Server Sent Events (SSE) Gateway.	1.24	卸载
✓	Snekeyaml API Plugin This plugin provides Snekeyaml for other plugins.	1.27.0	卸载
✓	SSH Build Agents Allows to launch agents over SSH, using a Java implementation	1.31.5	卸载

of the SSH protocol.

☒	SSH Credentials Allows storage of SSH credentials in Jenkins	1.18.1	卸载
☒	Structs Plugin Library plugin for DSL plugins that need names for Jenkins objects.	1.22	卸载
☒	Timestamp Adds timestamps to the Console Output	1.12	卸载
☒	Token Macro Plugin This plug-in adds reusable macro expansion capability for other plug-ins to use.	2.15	卸载
☒	Trilead API Plugin Trilead API Plugin provides the Trilead library to any dependent plugins in an easily update-able manner.	1.0.13	卸载
☒	Variant Plugin This user-invisible library plugin allows other multi-modal plugins to behave differently depending on where they run.	1.4	卸载
☒	Web for Blue Ocean Blue Ocean core	1.24.6	卸载
☒	Workspace Cleanup Plugin This plugin deletes the project workspace when invoked.	0.39	卸载

Jenkins 中文社区 REST API

9、邮件发送

- 系统配置 == 配置管理员邮箱

系统管理员邮件地址

o

534096094@qq.com

- 配置邮件发送的认证权限信息
 - 登录自己邮箱，开启POP3/SMTP邮件服务
 - 获取到自己的授权码 rkffwdqdyprccbcf
 - 配置并测试好邮件发送即可
- 邮件模板内容

```

1 emailxt body: '''<!DOCTYPE html>
2 <html>
3 <head>
4 <meta charset="UTF-8">
5 <title>${ENV, var="JOB_NAME"}-第${BUILD_NUMBER}次构建日志</title>
6 </head>
7
8 <body leftmargin="8" marginwidth="0" topmargin="8" marginheight="4"
9   offset="0">
10   <table width="95%" cellpadding="0" cellspacing="0" style="font-size: 11pt;
11     font-family: Tahoma, Arial, Helvetica, sans-serif">
12     <tr>
13       <td><font color="#CC0000">构建结果 - ${BUILD_STATUS}</font></td>
14     </tr>
15   </table>
16   <tr>
17     <td>各位同事，大家好，以下为${PROJECT_NAME }项目构建信息</td>
18   </tr>

```



```

17         <tr>
18             <td><br />
19             <b><font color="#0B610B">构建信息</font></b>
20             <hr size="2" width="100%" align="center" /></td>
21         </tr>
22         <tr>
23             <td>
24                 <ul>
25                     <li>项目名称 :  ${PROJECT_NAME}</li>
26                     <li>构建编号 :  第${BUILD_NUMBER}次构建</li>
27                     <li>触发原因:  ${CAUSE}</li>
28                     <li>构建状态:  ${BUILD_STATUS}</li>
29                     <li>构建日志:  <a
href="${BUILD_URL}console">${BUILD_URL}console</a></li>
30                     <li>构建  Url :  <a href="${BUILD_URL}">${BUILD_URL}</a></li>
31
                     <li>工作目录 :  <a
href="${PROJECT_URL}ws">${PROJECT_URL}ws</a></li>
32                     <li>项目  Url :  <a href="${PROJECT_URL}">${PROJECT_URL}</a>
</li>
33                 </ul>
34
35
36         <h4><font color="#0B610B">最近提交</font></h4>
37         <ul>
38             <hr size="2" width="100%" />
39             ${CHANGES_SINCE_LAST_SUCCESS, reverse=true, format="%c", changesFormat="<li>%d
[%a] %m</li>"}
40         </ul>
41         详细提交: <a href="${PROJECT_URL}changes">${PROJECT_URL}changes</a><br />
42
43             </td>
44         </tr>
45     </table>
46 </body>
47 </html>''', subject: '${ENV, var="JOB_NAME"}-第${BUILD_NUMBER}次构建日志', to:
'17512080612@163.com'
48
49
50     大家自己修改  to 的位置

```

实现 短信通知, 钉钉通知, 微信通知, xxxxx

3、高级使用

1、级联变量

1、基本选择 Active Choices Parameter

```
1 // groovy脚本
2 return ["hello", "world", "abc:selected"]
3
4
5 //fallback 定义错误返回
6 return "error"
7
```

2、引用级联 Active Choices Reactive Parameter

要指定之前参考参数

Referenced parameters ?

selected_branch

```
1 //如下
2 if ( selected_branch.equals("hello") ) {
3     return ["1", "2"]
4 }
5 if( selected_branch.equals("world") ){
6     return ["11", "22"]
7 }else{
8     return ["111", "222"]
9 }
10
11 //也可以执行shell
12
```

2、分支选择

3、自定义远程触发

Generic Webhook Trigger: 参照官网进行使用

三、 Jenkins On Docker

1

四、 Jenkins On Kubernetes

插件可能无法下载，可以手动去jenkins官网下载并上传

```
kubectcl cp /root/other/kubernetes-client-api.hpi devops/jenkins-0:/var/jenkins_home/plugins/
```

.hpi文件上传到jenkins的家目录下的plugins即可；不止k8s能用；什么方式拷贝过去都可以